



Banco de Dados

Prof. Elson de Abreu



Bloqueios

Bloqueios

- Bloqueio (lock) é um recurso utilizado pelo SGBD para que haja consistência das informações em um ambiente de transações concorrentes. Tal conjunto de regras pode ser configurado pelo usuário de acordo com suas necessidades, podendo indicar, por exemplo, o grau de isolamento desejado entre transações, definindo como será feito o bloqueio dos dados.
 - De forma mais didática, um lock pode ser considerado como uma sinalização em um objeto do banco (registro, tabela, etc), dizendo: “Alguém” está utilizando este recurso, aguarde a sua vez!.
 - No SQL Server é possível efetuar bloqueios por registros, páginas, grupo de registros (extent), tabelas ou até mesmo o todo o banco de dados. Normalmente o SQL Server efetua bloqueios no menor recurso disponível (bloqueios de linha), com o intuito de aumentar a concorrência (porém isso deve ser analisado com cautela, já que tal tipo de bloqueio pode gerar um *overhead* significativo, dependendo da aplicação).
 - O sistema gerenciador de bloqueios (lock manager) tem o objetivo de garantir que as requisições de bloqueios sejam feitas de forma eficiente.
- ✓ *Overhead: de maneira bem simples, é um custo adicional indesejável que deveria ou poderia ser evitado.*

Bloqueios

- Como exemplo, assumiremos a seguinte situação hipotética:
 - ✓ Suponha que uma transação “T1” esteja atualizando os registros de uma tabela “TAB” qualquer. No momento da atualização, uma leitura é feita por outra transação “T2” à mesma tabela “TAB”.
- ✓ Ficam então as perguntas:
 1. “T2” deve ficar aguardando até que “T1” finalize, para então ler os dados?
 2. “T2” deve ler apenas os dados não atualizados por “T1”, ignorando as possíveis atualizações?
 3. “T2” deve ler todos os dados, de forma indiscriminada, ignorando os bloqueios feitos por “T1”?

Níveis de isolamento de uma transação

- Em relação às perguntas do slide anterior, é possível assumir qualquer uma das três opções, bastando alterar o nível de isolamento de uma transação:

`SET TRANSACTION ISOLATION LEVEL` { *SERIALIZABLE* | *REPEATABLE READ* | *READ COMMITTED* | *READ UNCOMMITTED* }

- Esta é uma das duas maneiras de se alterar o nível de isolamento. A outra maneira é através de diretivas chamadas *hint's*.

✓ *O modo de isolamento "SNAPSHOT" não será abordado.*

Níveis de isolamento de uma transação

- **READ UNCOMMITTED:** Permite a leitura de dados não confirmados (“não comitados”), ignorando os bloqueios mantidos nas transações. Permite uma maior concorrência, porém com um nível de confiabilidade menor, devido a possibilidade de aparecimento de “registros fantasmas”.
- **READ COMMITTED:** Os processos de leitura só podem ser efetuados sobre dados confirmados (“comitados”). Bloqueios realizados em comandos SELECT são liberados no término do comando. Este é o nível de isolamento padrão.
- **REPEATABLE READ:** Incorpora as características de READ COMMITTED. Porém assegura que registros lidos não serão alterados por outros processos, garantido releituras idênticas. Bloqueios realizados em comandos SELECT são liberados somente ao término da transação.
- **SERIALIZABLE:** Incorpora as características de REPEATABLE READ, não permitindo, adicionalmente, que outras sessões efetuem inserções no conjunto de registros lidos. Este modo diminui drasticamente a concorrência.

Problema decorrente: Dirty Read

- Ocorre quando uma transação lê dados que foram alterados por uma outra transação, porém, a transação que alterou os dados ainda não “comitou” seu bloco de instruções. É conhecido como “leitura suja”.
- Suponha que existam 2 transações em execução (T1 e T2):
 - T1 atualiza a localização de um determinado departamento.
 - T2 faz uma “leitura suja” utilizando o hint *NOLOCK*, e vê o departamento atualizado por T1 com uma nova localização.
 - T1 efetua um *rollback*, desfazendo a alteração.
 - T2 não sabe, mas o registro nunca fora atualizado realmente, já que a transação T1 executou um *rollback*.

Dirty Read

```
-- EX. DIRTY READ  
-- TRANSACAO: T1
```

```
BEGIN TRAN
```

```
UPDATE DEPT  
  SET LOC = 'MIAMI'  
WHERE DEPTNO = 40
```

```
-- EX. DIRTY READ  
-- TRANSACAO: T2
```

```
SELECT * FROM DEPT (NOLOCK)  
WHERE LOC = 'MIAMI'
```

DEPTNO	NOME	LOC
40	OPERACIONAL	MIAMI

(1 row(s) affected)

```
-- EX. DIRTY READ  
-- TRANSACAO: T1
```

```
BEGIN TRAN
```

```
UPDATE DEPT  
  SET LOC = 'MIAMI'  
WHERE DEPTNO = 40
```

```
ROLLBACK
```

Exemplo com o bd “Empresa”

T2 não sabe, mas o
valor não foi atualizado!

Problema decorrente: non-repeatable read

- Ocorre quando uma transação lê um conjunto de dados, depois mais tarde, ao ler novamente tais dados, descobre que os mesmos mudaram. Também conhecido como “leitura que não pode ser repetida”.

Problema decorrente: Lost Updates

- Um Lost Update ocorre quando duas mudanças diferentes são escritas no banco de dados, e a segunda mudança causa a perda da primeira (sobreposição).
- Suponha que existam 2 transações (T1 e T2):
 - T1 faz uma atualização da localização de um departamento.
 - T2 também faz uma atualização e sobrescreve a informação atualizada por T1.
 - O dado atualizado por T1 foi perdido.

Lost Updates

```
-- EX. LOST UPDATE  
-- TRANSACAO: T1
```

```
UPDATE DEPT  
  SET LOC = 'LOS ANGELES'  
WHERE DEPTNO = 40
```

```
SELECT * FROM DEPT
```

DEPTNO	NOME	LOC
10	CONTABILIDADE	NEW YORK
20	PESQUISA	DALLAS
30	VENDA	CHICAGO
40	OPERACIONAL	LOS ANGELES

```
-- EX. LOST UPDATE  
-- TRANSACAO: T2
```

```
UPDATE DEPT  
  SET LOC = 'NEW YORK'  
WHERE DEPTNO = 40
```

```
SELECT * FROM DEPT
```

DEPTNO	NOME	LOC
10	CONTABILIDADE	NEW YORK
20	PESQUISA	DALLAS
30	VENDA	CHICAGO
40	OPERACIONAL	NEW YORK

Dica para resolver Lost Updates

- Normalmente se utiliza um campo de controle do tipo *timestamp* ou *uniqueidentifier* para resolver o problema.

Observe a nova estrutura da tabela DEPT:

- ✓ Adicione o novo campo de controle “CONCORRENCIA”, do tipo *uniqueidentifier* (após adicionar o campo, dê um update na tabela e depois marque o campo como *not null*).

Table - dbo.DEPT*			
	Column Name	Data Type	Allow Nulls
	DEPTNO	int	☐
	NOME	varchar(14)	☒
	LOC	varchar(13)	☒
	CONCORRENCIA	uniqueidentifier	☐

```
UPDATE DEPT
SET CONCORRENCIA = NEWID ()
```

```
GO
```

```
SELECT * FROM DEPT
GO
```

Exemplo com o bd “Empresa”

```
{4 row(s) affected}
DEPTNO    NOME          LOC          CONCORRENCIA
-----
10        CONTABILIDADE NEW YORK     3A1A8629-4974-4422-87BB-F4950AA5B94A
20        PESQUISA      DALLAS      D177B94C-C1FF-408E-B56F-7496719252E4
30        VENDA        CHICAGO     9A5ED75A-1D2C-48FC-9F2A-AC54D5BF4FFD
40        OPERACIONAL  BOSTON      D3291DA4-332B-4AC5-A09D-989A036B2DF7
```

```
{4 row(s) affected}
```

Dica para resolver Lost Updates

```
DECLARE @DEPTNO INT,  
        @NOME   VARCHAR(14),  
        @LOC    VARCHAR(13),  
        @CHAVE  UNIQUEIDENTIFIER  
  
SELECT @DEPTNO = DEPTNO, @NOME = NOME, @LOC = LOC, @CHAVE = CONCORRENCIA  
FROM DEPT  
WHERE DEPTNO = 10  
  
UPDATE DEPT  
SET LOC = 'LOS ANGELES'  
WHERE DEPTNO = @DEPTNO  
AND CONCORRENCIA = @CHAVE  
  
IF @@ROWCOUNT = 0  
PRINT 'REGISTRO JÁ ATUALIZADO POR OUTRA TRANSACAO !'  
  
GO
```

1. As transações fazem a leitura do mesmo registro, obtendo o registro com a chave de concorrência **9CD8C757-4377-4E82-838D-A4EDAFD6AC28 ***, armazenando tal valor em uma variável.
2. A Transação 2 faz o *update* antes da Transação 1.
3. A Transação 1 não conseguiu atualizar as informações pois a chave de concorrência ficou obsoleta, sendo forçada a fazer uma nova leitura dos dados e consequentemente uma nova chave de concorrência.
4. Obs.: * **Valor de exemplo.**

Problema decorrente: Phantom Read

- Este problema ocorre quando um novo registro é inserido por uma transação T1, porém ainda não “comitado”. Logo após a inserção, uma transação T2 faz a leitura da mesma tabela utilizando o hint NOLOCK. A transação T2 irá enxergar este novo registro, porém a transação T1 pouco depois desfaz a inserção (rollback), “sumindo” com o registro visto por T2.

Transação 1

```
BEGIN TRAN
```

```
INSERT INTO DEPT (DEPTNO, NOME, LOC, CONCORRENCIA)  
VALUES (50, 'TI', 'BH', NEWID())
```

```
ROLLBACK
```

```
SELECT * FROM DEPT
```

DEPTNO	NOME	LOC	CONCORRENCIA
10	CONTABILIDADE	LOS ANGELES	A8F16F17-18DE-4340-B8AC-01E0A32C7154
20	PESQUISA	DALLAS	2D10794F-1A7F-4505-ADB4-8105E23F4BE7
30	VENDA	CHICAGO	9CF293A6-A644-4682-865A-5D5CD270EC0B
40	OPERACIONAL	NEW YORK	EB71D265-5515-400B-B3FE-34778A8D84B3

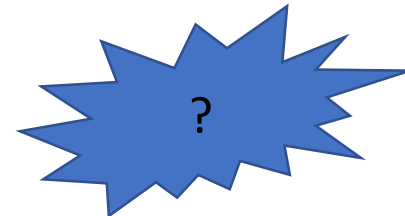
(4 row(s) affected)

Transação 2

```
SELECT * FROM DEPT (NOLOCK)
```

DEPTNO	NOME	LOC	CONCORRENCIA
10	CONTABILIDADE	LOS ANGELES	A8F16F17-18DE-4340-B8AC-01E0A32C7154
20	PESQUISA	DALLAS	2D10794F-1A7F-4505-ADB4-8105E23F4BE7
30	VENDA	CHICAGO	9CF293A6-A644-4682-865A-5D5CD270EC0B
40	OPERACIONAL	NEW YORK	EB71D265-5515-400B-B3FE-34778A8D84B3
50	TI	BH	86F5A6CA-D710-4D92-BCB5-51AF5E282C36

(5 row(s) affected)



Exemplo com o bd “Empresa”

O uso de hints

- O comportamento do bloqueio está diretamente associado ao nível de isolamento da transação. Especificar para um comando o nível de isolamento diferente do adotado pela transação corrente através dos *hints* (diretivas declaradas junto aos comandos de manipulação de dados) é uma prática comum.

Hints: alterando a granularidade das transações

- **ROWLOCK** : Efetua bloqueio de linha na execução do comando. Utilizando este hint não será possível passar para os demais níveis de granularidade.
- **PAGLOCK** : Efetua bloqueio de página na execução do comando.
- **TABLOCK** : Efetua bloqueio de tabela na execução do comando.
- **TABLOCKX** : Efetua bloqueio de tabela (exclusivo) na execução do comando.

Hints: alterando o nível de isolamento da transações

- **HOLDLOCK** : Prolonga a duração do bloqueio até o término da transação. Utilizar com o comando “SELECT”.
(Lembra-se do REPEATABLEREAD ?)
- **NOLOCK** : Leitura em dados não comitados (dirty read).
- **READUNCOMMITTED** : Idem “nolock”
- **READCOMMITTED** : Trabalha com isolamento “READCOMMITTED”.
- **READPAST** : Instrui a query a pular os registros bloqueados. Utilizar com o comando “SELECT”.
- **REPEATABLEREAD** : Segue as características do isolamento de mesmo nome.
- **SERIALIZABLE** : Segue as características do isolamento de mesmo nome.

Outros tipos de Hints

- **UPDLOCK** : Utilizado em comandos SELECT para restringir pesquisas concorrentes. Apenas uma transação pode obter o recurso com este bloqueio.
- **XLOCK** : Enquanto UPDLOCK permite a leitura dos dados por outras transações (SELECT's sem a utilização de hints), XLOCK impõe bloqueio para as linhas lidas, impedindo qualquer operação, nem mesmo a leitura.

Dicas para minimizar bloqueios

- Utilize transações curtas. Isso aumenta o grau de concorrência e reduz o tempo para desfazer, caso seja preciso.
- Evite transações aninhadas.
- Elimine o uso de locks quando não for necessário. Por exemplo, transações somente de leitura.
- Gere valores auto-incrementáveis com a propriedade *identity*, pois é mais eficiente.
- Transações de longa duração devem usar locks a nível de tabela a fim de evitar deadlocks (situações de impasse).

Vamos simular...

- Dirty Read: siga o exemplo do slide 7.
- Phantom Read: siga o exemplo do slide 13.
- Lost Updates: siga a dica dos slides 11 e 12.
- ReadPast: siga o mesmo exemplo de Phantom Read, porém substitua o hint por *READPAST*.
- TabLockX (copie o exemplo de HoldLock, substitua o hint e exclua a cláusula *where* dos comandos).
- HoldLock (alterando o nível de isolamento):

-- TRANSACAO 1

BEGIN TRAN

SELECT * FROM DEPT (HOLDLOCK)
WHERE DEPTNO = 10

-- TRANSACAO 2

BEGIN TRAN

SELECT * FROM DEPT
WHERE DEPTNO = 10

UPDATE DEPT
SET LOC = 'ABC'
WHERE DEPTNO = 10